Stage 0.5 firmware — Pi Zero 2W

Pi-side C++ that drives the Stage 0.5 board (4×1 Q8.8.8 dot product) and runs the three bring-up experiments described in `../STAGE0_5.md`.

Zero external library dependencies. Just POSIX, the standard C++ library, and Linux kernel device interfaces (`/dev/spidev*`, `/dev/i2c-*`, `/sys/class/gpio`). One source file, one Makefile, no `apt install` required beyond a stock Pi OS image.

One-time Pi setup

This assumes a fresh Pi OS Lite image on the Pi Zero 2W.

```
sudo raspi-config
#   Interface Options → SPI  → Enable
#   Interface Options → I2C  → Enable
#   reboot
```

Verify both interfaces are present:

```
ls /dev/spidev0.0    # SPI0 device — should exist
ls /dev/i2c-1        # I²C1 device — should exist
```

Install build tools (only `g++` and `make` are needed; both are usually already in `build-essential`):

```
sudo apt update && sudo apt install -y build-essential
```

You can scan I²C to confirm the DAC and ADC are answering:

```
sudo apt install -y i2c-tools
sudo i2cdetect -y 1
#   Expected: 0x48 (ADS1115)  and  0x60 (MCP4728)
```

If either address is missing, the corresponding chip is not seeing the SDA/SCL bus — check the 4.7 kΩ pull-ups and your soldering before trying the firmware.

Build & run

```
cd analog-sidequest/stage0_5-firmware
make            # builds ./stage0_5
make run        # runs all three experiments (sudo)
```

Per-experiment runs:

```
make exp1       # slice-summer linearity check
make exp2       # full Q8.8.8 dot-product vs software reference
make exp3       # stochastic-rounded write + SPI readback
```

Each experiment prints PASS / FAIL on a per-step and final basis. Exit code is 0 if all requested experiments pass, 1 otherwise.

Why this needs `sudo`

Two things require root on a stock Pi OS image:

1. **Writing to `/sys/class/gpio/export`.** Sysfs GPIO export is gated by the `gpio` group; a stock user account isn't in it.
2. **Opening `/dev/spidev0.0` with `SPI_NO_CS` mode.** The kernel's spidev driver allows this only with elevated privileges by default.

Both can be configured away with udev rules and group adjustments, but the bring-up board is not the place to fight Linux permissions. Run as root, get the experiment done, harden later.

Pin map (matches `../STAGE0_5.md`)

```
Pi Zero 2W          ↔   Stage 0.5 board
──────────────────────────────────────────────────────
GPIO 10  (pin 19)   →   SPI0 MOSI    →   all 6 MCP4251 SDI
GPIO  9  (pin 21)   ←   SPI0 MISO    ←   all 6 MCP4251 SDO  (tristate)
GPIO 11  (pin 23)   →   SPI0 SCLK    →   all 6 MCP4251 SCK
GPIO  8  (pin 24)   →   74HC138 A    (chip-address LSB)
GPIO  7  (pin 26)   →   74HC138 B
GPIO  5  (pin 29)   →   74HC138 C    (chip-address MSB)
```

```
GPIO  6  (pin 31)   →   74HC138 G2A  (active-low decoder enable)
GPIO  2  (pin  3)   ↔   I²C1 SDA     →   MCP4728 + ADS1115
GPIO  3  (pin  5)   ↔   I²C1 SCL     →   MCP4728 + ADS1115
3.3 V    (pin  1)   →   digital VDD rail (MCP4251s, 74HC138)
5 V      (pin  2)   →   AMS1117 LDO input → analog 3.3 V rail
GND      (pin  6, 14, 30, ...)
```

What each experiment does

Experiment 1 — slice-summer linearity

For each of the three slices (MSB, middle, LSB) in turn:

1. Sets all four wipers in that slice to mid-scale (tap 128 of 256).
2. Drives each input one at a time at 1.024 V (half-rail of the 2.048 V DAC reference); reads the slice's mid-point on its dedicated debug ADC channel (ch.1 = V_msb, ch.2 = V_mid, ch.3 = V_lsb).
3. Drives all four inputs together; output should be approximately 4× a single-input step.

Pass criterion: each per-input step within ±1 % of the four-input mean, and the sum-of-four output within ±1 % of 4× mean.

If this fails, the topology is broken at the slice level — there is no point running Experiment 2 until each slice summer is working independently.

Experiment 2 — full Q8.8.8 dot product vs software reference

Programs random Q8.8.8 weights in [0, 1] and drives random inputs in [0, 1.024 V] across 50 trials. For each trial, compares the measured V_out to the software dot-product computed on the same weights and inputs (the simulator's reconstruction math).

Pass criterion: RMS error / RMS signal ≤ 0.1 % across the 50 trials. (The Stage 0 noise budget is $\sigma = 0.0003$ per-MAC × $\sqrt{4} \approx 0.06$ %; 0.1 % gives bench-measurement margin.)

The first trial is used to learn the analog scale factor (volts per unit of dot-product) — Stage 0.5 does not require knowing the absolute op-amp gain in advance; it learns it on the bench.

Experiment 3 — stochastic-rounded write + SPI readback

Initialises all four weights to 0.5. Applies a small $\Delta w = +0.0001$ update through `stochastic_update` (the same logic as `../stage0-sim/analog_sim.cpp`'s `Tile::write_delta`). Every 100 steps, reads back the wiper register from each MCP4251 and verifies that the hardware tap values match the firmware's tracked Q8.8.8 state.

Pass criterion: zero readback mismatches across 1000 steps × 4 weights × 3 slices = 12,000 readback checks. Any mismatch indicates either a dropped SPI write, a decoder glitch, or a bus-contention bug during readback — all things we want to find here, on a $25 PCB, not on the $500 Stage 1 board.

Passing Experiment 3 means the firmware path that Stage 1 will run unmodified — write a stochastically-rounded weight delta over Q8.8.8 slices, expect the analog cells to land where the simulator predicted — works in physics on this board. That is what Stage 0.5 sets out to prove.

Where to look when something fails

- **Experiment 1 fails on one slice but not others.** That slice's feedback resistor or one of its four digipots is wrong. Probe the slice op-amp's output with a scope while toggling each input alone.
- **Experiment 1 passes but Experiment 2 fails.** The combiner stages are off. Probe the first combiner's output (op-amp D in the schematic) and verify (V_msb + V_mid/256). If that's right, the problem is in the second combiner.
- **Experiment 3 fails on every readback.** The MISO bus is floating or the kernel is reading garbage. Add (or check) the 10 kΩ pull-up to 3.3 V on MISO.
- **Experiment 3 fails sporadically.** Decoder glitch — address bits are changing while ENABLE is asserted. Inspect `select_chip` to confirm ENABLE goes high *before* address bits change and stays high until they're settled.
- **Nothing works at all (no DAC output, no ADC reading).** Run `sudo i2cdetect -y 1` and `sudo dmesg | tail -20`. If the I²C devices aren't showing up, the SDA/SCL routing is broken or the pull-ups are wrong. If the SPI device is missing, raspi-config didn't take effect.